SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA02 **COURSE NAME: C Programming**

COURSE OUTCOME COVERED

CO1: *Construct structured C programs using constants, variables, data types, and preprocessor directives to address basic computational tasks. (BL2,BL3)*

Assessment Tool Weightage: 20%

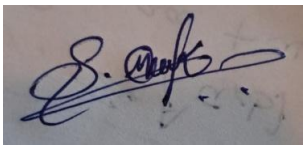
QUESTIONS: **Total Marks:100**

Annexure A

EXPERIMENTS

Code of Conduct:

I Shaik Mubarak(192425194)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

Experiments

1. Write a C program to:
- Display intermediate values using a macro DEBUG

Perform:

- Run with DEBUG = 1
- Run with DEBUG = 0

Record:

- Output differences

Conclude:

- How macros control program behavior

Q2. Data Type Experiment

Write a program to calculate average of 3 numbers using:

- int
- float

Perform:

- Run both versions

Observe:

- Difference in output

Conclude:

- Effect of data types on accuracy

Q3. Constant vs Variable Experiment

Write two versions of a program to compute the circumference of a circle:

- One using #define for constants
- One using variables

Perform:

- Change values and run

Observe:

- Which is easier to modify

Conclude:

- Advantage of macros

Q4. Type Casting Experiment

Write a program to compute average:

- Without type casting
- With type casting

Observe:

- Output difference

Conclude:

- Why type casting is required

Q5. Input Validation Experiment

Write a program to:

- Accept input within a defined range using constants

Perform:

- Test with valid and invalid inputs

Observe:

- Program response

Conclude:

- Importance of validation

Q6. Macro-Based Array Size

Write a program where array size is defined using macro

Perform:

- Change size and re-run

Observe:

- Output behavior

Conclude:

- Benefit of compile-time configuration

Q7. Feature Enable/Disable Experiment

Write a program to read amount to be paid with

- Optional discount (if the amount is greater than 1000, give 5% discount) using macro

Perform:

- Enable and disable feature

Observe:

- Change in total value

Conclude:

- Flexibility of preprocessor directives

Q8. Precision Experiment

Write a program to calculate area of circle

Perform:

- Use float and double

Observe:

- Precision difference

Conclude:

- Best data type choice

Q9. Overflow Experiment

Write a program to:

- Store large values in different data types

Perform:

- Test with large inputs

Observe:

- Overflow behavior

Conclude:

- Limitations of data types

Q10. Multi-Case Testing

Write a program to:

- Convert temperature (Kelvin to Celcius, Kelvin to Farenheit)

Perform:

- Test with 3 different inputs

Observe:

- Output correctness

Conclude:

- Program reliability

Q11. Replace Hardcoding

Write a program to swap two values. Hard code the values.

Modify:

- Replace with macros

Observe:

- Code readability

Conclude:

- Importance of symbolic constants

Q12. Control Structure Experiment

Write a program to read three marks and calculate the grade based on the following constraints:

- if the student fails in any one subject his grade is displayed as ‘F’.
- if the average is > 90, display ‘S’
- if the average is > 80, display ‘A’
- if the average is > 70, display ‘B’

Perform:

- Test boundary values

Observe:

- Output correctness

Conclude:

- Role of conditions

Q13. Step-by-Step Output Tracking

Write a program to calculate the electricity consumption.

Add:

- Print statements using macro

Observe:

- Intermediate results

Conclude:

- Debugging benefits

Q14. Minimal Memory Design

Write a program that reads cost of an item, number of items bought and calculate the total amount to be paid:

Apply:

- smallest suitable data types

Perform:

- Compare with normal version

Observe:

- Efficiency

Conclude:

- Importance of data selection

Q15. Program Modification Task

Write a basic program to find the celsius using Farenheit

Modify:

- Add macros
- Change data types

Observe:

- Output changes

Conclude:

- Impact of design choices

Solutions:

Experiment 1: Display Intermediate Values Using Macro DEBUG

Aim

To demonstrate how the DEBUG macro controls the display of intermediate values in a C program.

Algorithm

1. Start the program.
2. Define the DEBUG macro.
3. Read two integer values.
4. Calculate their sum.
5. If DEBUG is 1, display intermediate values.
6. Display the final sum.
7. Stop.

Program:

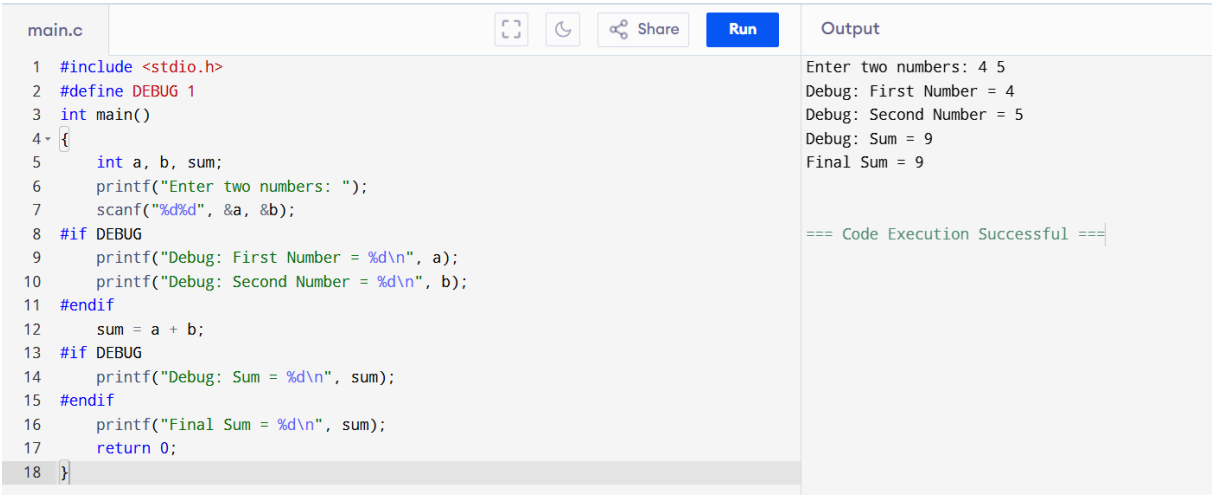
Run with Debug = 1

Code:

```
#include <stdio.h>
#define DEBUG 1

int main()
{
   int a, b, sum;

   printf("Enter two numbers: ");
   scanf("%d%d", &a, &b);

#if DEBUG
   printf("Debug: First Number = %d\n", a);
   printf("Debug: Second Number = %d\n", b);
#endif

   sum = a + b;

#if DEBUG
   printf("Debug: Sum = %d\n", sum);
#endif
```

```
    printf("Final Sum = %d\n", sum);

    return 0;
}
```

Output



When debug = 0

Code:

```
#include <stdio.h>

#define DEBUG 0

int main()
{
    int a, b, sum;

    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);

#if DEBUG
    printf("Debug: First Number = %d\n", a);
    printf("Debug: Second Number = %d\n", b);
#endif

    sum = a + b;

#if DEBUG
    printf("Debug: Sum = %d\n", sum);
#endif

    printf("Final Sum = %d\n", sum);

    return 0;
}
```

Output:

Result

Thus, the program was executed successfully and demonstrated that the DEBUG macro controls whether intermediate values are displayed.

Experiment 2: Data Type Experiment

Aim

To calculate the average of three numbers using int and float data types and observe the difference in accuracy.

Algorithm

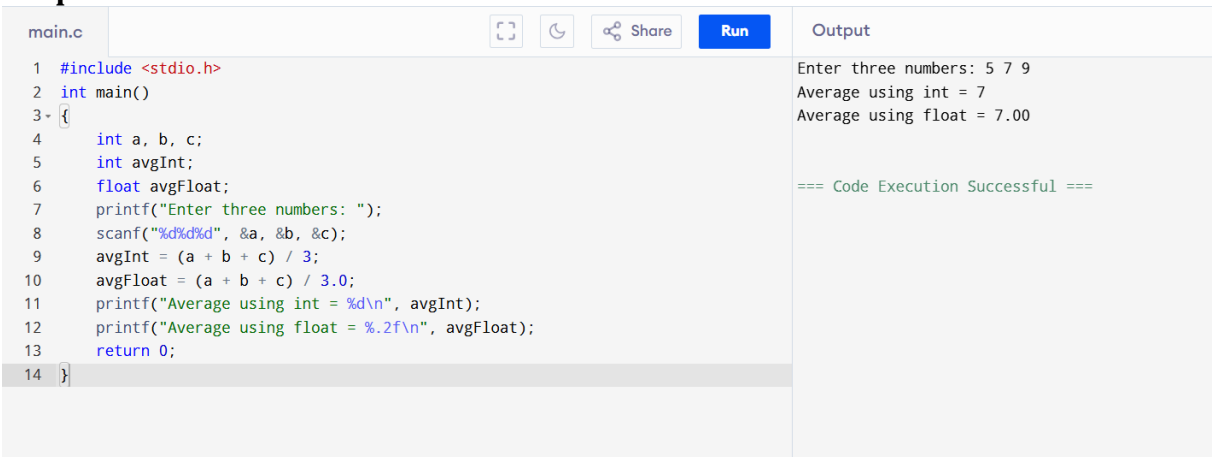
1. Start the program.
2. Read three numbers.
3. Calculate the average using integer data type.
4. Calculate the average using float data type.
5. Display both averages.
6. Stop.

Program

```
#include <stdio.h>
int main()
{
  int a, b, c;
  int avgInt;
  float avgFloat;
  printf("Enter three numbers: ");
  scanf("%d%d%d", &a, &b, &c);
  avgInt = (a + b + c) / 3;
  avgFloat = (a + b + c) / 3.0;
```

```
    printf("Average using int = %d\n", avgInt);
    printf("Average using float = %.2f\n", avgFloat);
    return 0;
}
```

Output:



Result

Thus, the program was executed successfully. It shows that the float data type provides a more accurate average than the int data type.

Experiment 3: Constant vs Variable Experiment

Aim

To compute the circumference of a circle using #define constants and variables, and compare their usage.

Algorithm

1. Start the program.
2. Define the value of PI using #define.
3. Read the radius.
4. Calculate the circumference using the macro constant.
5. Assign PI to a variable.
6. Calculate the circumference using the variable.
7. Display both results.
8. Stop.

Program

```
 #include <stdio.h>
#define PI 3.14
int main()
{
   float radius, pi, c1, c2;
   printf("Enter the radius: ");
```

```
    scanf("%f", &radius);
    c1 = 2 * PI * radius;
    pi = 3.14;
    c2 = 2 * pi * radius;
    printf("Circumference using #define = %.2f\n", c1);
    printf("Circumference using variable = %.2f\n", c2);
    return 0;
}
```

Output:

Result

Thus, the program was executed successfully. The circumference was calculated using both a macro constant and a variable. Using #define makes constants easier to modify and improves code readability.

Experiment 4: Type Casting Experiment

Aim

To compute the average of three numbers with and without type casting and observe the difference in the output.

Algorithm

1. Start the program.
2. Read three integer values.
3. Calculate the average without type casting.
4. Calculate the average with type casting.
5. Display both results.
6. Stop.

Program

```
#include <stdio.h>

int main()
{
   int a, b, c;
```

```
    float avg1, avg2;

    printf("Enter three numbers: ");
    scanf("%d%d%d", &a, &b, &c);

    avg1 = (a + b + c) / 3;
    avg2 = (float)(a + b + c) / 3;

    printf("Average without type casting = %.2f\n", avg1);
    printf("Average with type casting = %.2f\n", avg2);

    return 0;
}
```

Output

Result

Thus, the program was executed successfully. Type casting produced a more accurate average by preventing integer division.

Experiment 5: Input Validation Experiment

Aim

To write a C program that accepts input within a defined range using constants and validates the input.

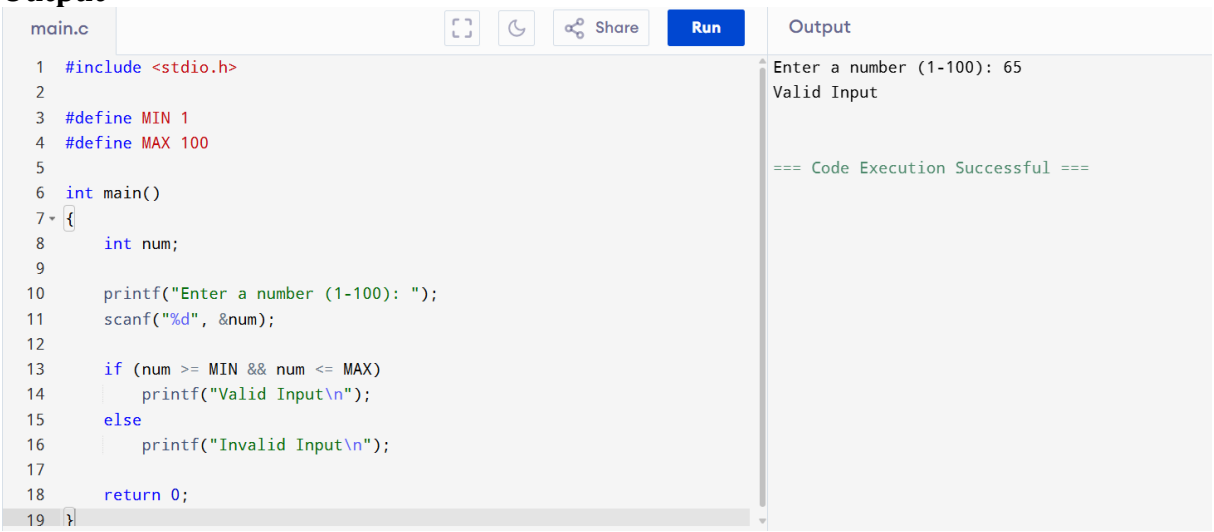
Algorithm

1. Start the program.
2. Define the minimum and maximum range using #define.
3. Read the input value.
4. Check whether the value lies within the specified range.
5. If valid, display "Valid Input".
6. Otherwise, display "Invalid Input".
7. Stop.

Program

```
#include <stdio.h>

#define MIN 1
#define MAX 100

int main()
{
   int num;

   printf("Enter a number (1-100): ");
   scanf("%d", &num);

   if (num >= MIN && num <= MAX)
      printf("Valid Input\n");
   else
      printf("Invalid Input\n");

   return 0;
}
```

Output



Result

Thus, the program was executed successfully and validated the input using predefined constants.

Experiment 6: Macro-Based Array Size

Aim

To write a C program where the array size is defined using a macro and observe the effect of changing the array size.

Algorithm

1. Start the program.
2. Define the array size using the #define macro.
3. Declare an array using the macro.
4. Read the array elements.
5. Display the array elements.
6. Stop.

Program

```
#include <stdio.h>

#define SIZE 5

int main()
{
   int arr[SIZE], i;

   printf("Enter %d elements:\n", SIZE);

   for(i = 0; i < SIZE; i++)
   {
      scanf("%d", &arr[i]);
   }

   printf("Array elements are:\n");

   for(i = 0; i < SIZE; i++)
   {
      printf("%d ", arr[i]);
   }

   return 0;
}
```

Output

Result

Thus, the program was executed successfully. The array size was controlled using a macro, making it easy to change the array size at compile time.

Experiment 7: Feature Enable/Disable Experiment

Aim

To write a C program to calculate the amount to be paid with an optional 5% discount using a macro.

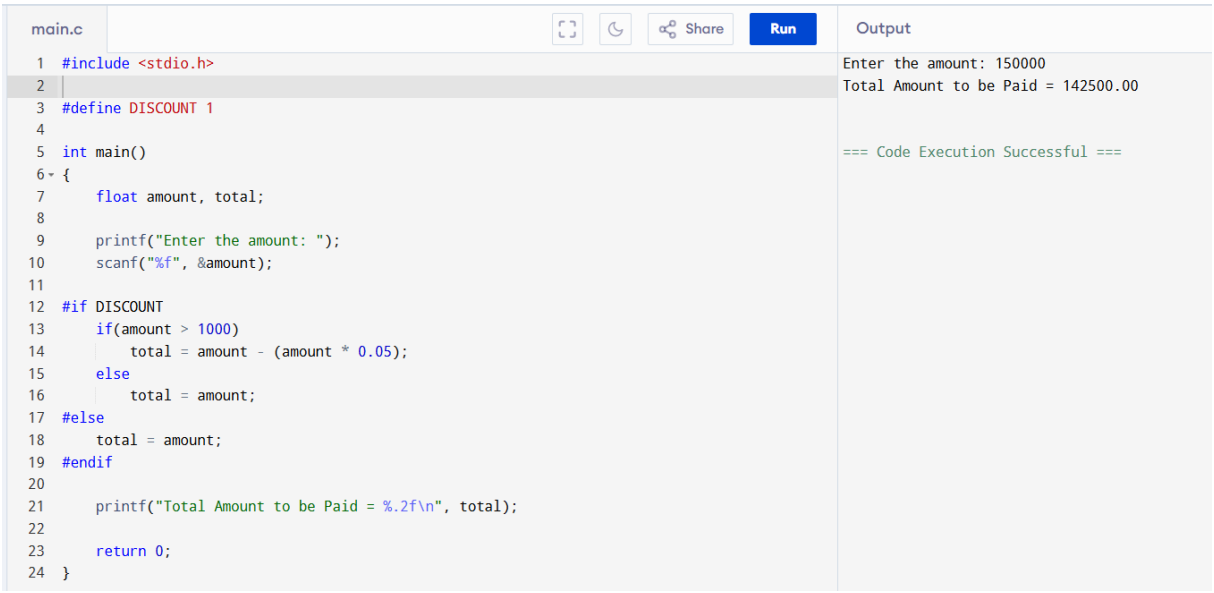
Algorithm

1. Start the program.
2. Define a macro to enable or disable the discount feature.
3. Read the amount.
4. If the discount feature is enabled and the amount is greater than 1000, calculate a 5% discount.
5. Display the final amount.
6. Stop.

Program

```
#include <stdio.h>

#define DISCOUNT 1

int main()
{
   float amount, total;

   printf("Enter the amount: ");
   scanf("%f", &amount);

#if DISCOUNT
   if(amount > 1000)
      total = amount - (amount * 0.05);
   else
      total = amount;
#else
   total = amount;
#endif

   printf("Total Amount to be Paid = %.2f\n", total);

   return 0;
}
```

Output



Result

Thus, the program was executed successfully. The macro enabled and disabled the discount feature without changing the program logic.

Experiment 8: Precision Experiment

Aim

To write a C program to calculate the area of a circle using float and double data types and compare their precision.

Algorithm

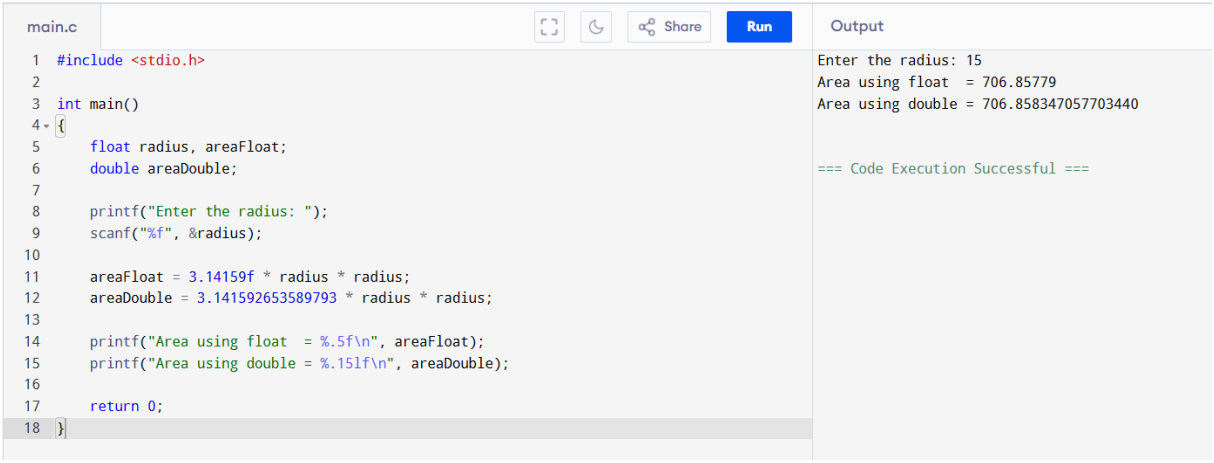
1. Start the program.
2. Read the radius of the circle.
3. Calculate the area using the float data type.
4. Calculate the area using the double data type.
5. Display both results.
6. Stop.

Program

```
#include <stdio.h>
int main()
{
   float radius, areaFloat;
   double areaDouble;
   printf("Enter the radius: ");
   scanf("%f", &radius);
   areaFloat = 3.14159f * radius * radius;
   areaDouble = 3.141592653589793 * radius * radius;
   printf("Area using float  = %.5f\n", areaFloat);
```

```
    printf("Area using double = %.15lf\n", areaDouble);
    return 0;
}
```

Output



Result

Thus, the program was executed successfully. The double data type produced more precise results than the float data type.

Experiment 9: Overflow Experiment

Aim

To write a C program to store large values in different data types and observe the overflow behavior.

Algorithm

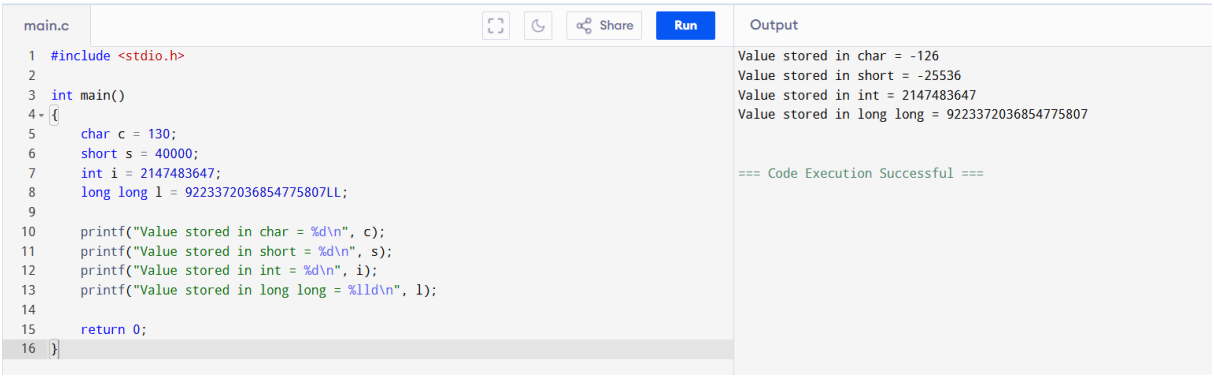
1. Start the program.
2. Declare variables of different data types (char, short, int, and long long).
3. Assign large values to each variable.
4. Display the stored values.
5. Observe the overflow in smaller data types.
6. Stop.

Program

```
#include <stdio.h>
int main()
{
    char c = 130;
    short s = 40000;
    int i = 2147483647;
    long long l = 92233720368547758O7LL;
    printf("Value stored in char = %d\n", c);
    printf("Value stored in short = %d\n", s);
    printf("Value stored in int = %d\n", i);
    printf("Value stored in long long = %lld\n", l);
```

```
    return 0;
}
```

Output



Note: The exact overflow values for char and short may vary depending on the compiler and system.

Result

Thus, the program was executed successfully. It demonstrated that storing values beyond the capacity of a data type causes overflow, leading to incorrect results.

Experiment 10: Multi-Case Testing

Aim

To write a C program to convert temperature from Kelvin to Celsius and Kelvin to Fahrenheit and test it with different inputs.

Algorithm

1. Start the program.
2. Read the temperature in Kelvin.
3. Convert Kelvin to Celsius using C = K - 273.15.
4. Convert Kelvin to Fahrenheit using F = (C × 9/5) + 32.
5. Display the converted temperatures.
6. Stop.

Program

```
#include <stdio.h>
int main()
{
   float kelvin, celsius, fahrenheit;
   printf("Enter temperature in Kelvin: ");
   scanf("%f", &kelvin);
   celsius = kelvin - 273.15;
   fahrenheit = (celsius * 9 / 5) + 32;
   printf("Temperature in Celsius = %.2f\n", celsius);
   printf("Temperature in Fahrenheit = %.2f\n", fahrenheit);
```

```
    return 0;
}
```

Output

Result

Thus, the program was executed successfully and correctly converted Kelvin temperature to Celsius and Fahrenheit for different test cases.

Experiment 11: Replace Hardcoding

Aim

To write a C program to swap two values using hardcoded values and then replace them with macros to improve code readability.

Algorithm

1. Start the program.
2. Define two values using macros.
3. Assign the macro values to two variables.
4. Display the values before swapping.
5. Swap the values using a temporary variable.
6. Display the values after swapping.
7. Stop.

Program

```
#include <stdio.h>
#define A 10
#define B 20
int main()
{
   int x = A, y = B, temp;
   printf("Before Swapping:\n");
   printf("x = %d, y = %d\n", x, y);
   temp = x;
   x = y;
   y = temp;
```

```
    printf("After Swapping:\n");
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

Output

Result

Thus, the program was executed successfully. Replacing hardcoded values with macros improved the readability and maintainability of the program.

Experiment 12: Control Structure Experiment

Aim

To write a C program to read three subject marks and calculate the grade based on the given conditions.

Algorithm

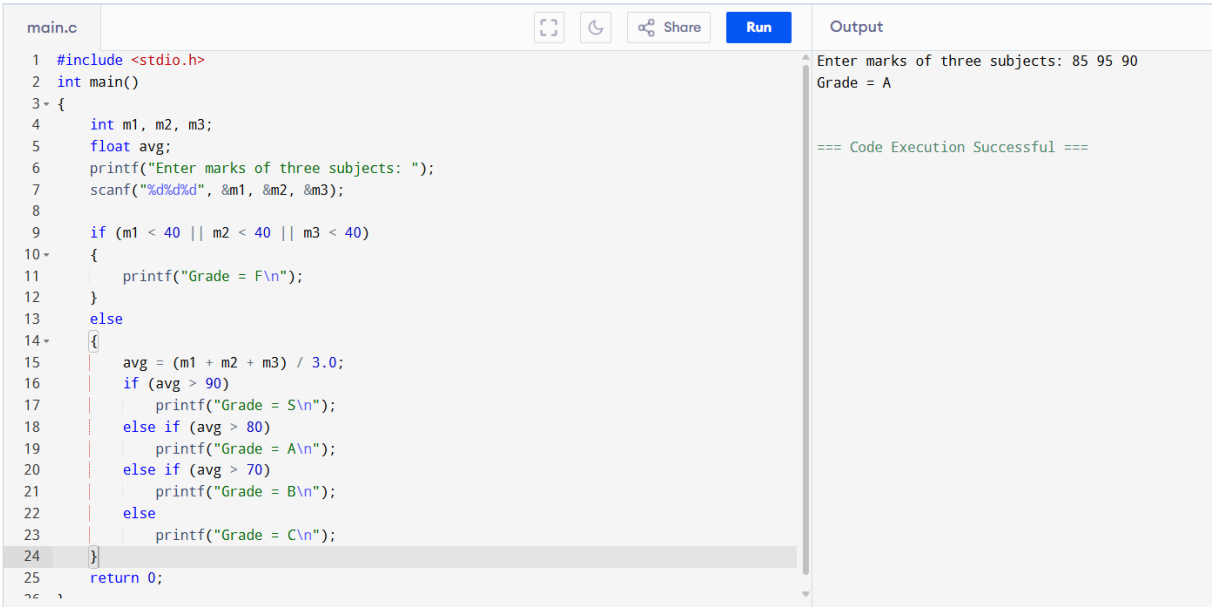
1. Start the program.
2. Read the marks of three subjects.
3. Check if the student has failed in any subject (marks < 40).
4. If failed, display grade **F**.
5. Otherwise, calculate the average.
6. Display **S** if average > 90.
7. Display **A** if average > 80.
8. Display **B** if average > 70.
9. Otherwise, display **C**.
10. Stop.

Program

```
#include <stdio.h>

int main()
{
```

```
    int m1, m2, m3;
    float avg;
    printf("Enter marks of three subjects: ");
    scanf("%d%d%d", &m1, &m2, &m3);
    if (m1 < 40 || m2 < 40 || m3 < 40)
    {
        printf("Grade = F\n");
    }
    else
    {
        avg = (m1 + m2 + m3) / 3.0;
        if (avg > 90)
            printf("Grade = S\n");
        else if (avg > 80)
            printf("Grade = A\n");
        else if (avg > 70)
            printf("Grade = B\n");
        else
            printf("Grade = C\n");
    }
    return 0;
}
```

Output



Result

Thus, the program was executed successfully and the grade was displayed correctly based on the given conditions.

Experiment 13: Step-by-Step Output Tracking
Aim

To write a C program to calculate electricity consumption and display intermediate results using a macro for debugging.

Algorithm

1. Start the program.
2. Define the DEBUG macro.
3. Read the number of units consumed.
4. Calculate the electricity bill based on the rate per unit.
5. If DEBUG is enabled, display intermediate values.
6. Display the total electricity bill.
7. Stop.

Program

```
#include <stdio.h>
#define DEBUG 1
int main()
{
   int units;
   float rate = 5.0, bill;
   printf("Enter electricity units consumed: ");
   scanf("%d", &units);
#if DEBUG
   printf("Debug: Units = %d\n", units);
   printf("Debug: Rate per unit = %.2f\n", rate);
#endif
   bill = units * rate;
#if DEBUG
   printf("Debug: Calculated Bill = %.2f\n", bill);
#endif
   printf("Total Electricity Bill = %.2f\n", bill);
   return 0;
}
```

Output

main.c

Share Run

```c
#include <stdio.h>
#define DEBUG 1
int main()
{
    int units;
    float rate = 5.0, bill;
    printf("Enter electricity units consumed: ");
    scanf("%d", &units);
#if DEBUG
    printf("Debug: Units = %d\n", units);
    printf("Debug: Rate per unit = %.2f\n", rate);
#endif
    bill = units * rate;
#if DEBUG
    printf("Debug: Calculated Bill = %.2f\n", bill);
#endif
    printf("Total Electricity Bill = %.2f\n", bill);
    return 0;
}
```

Output

```
Enter electricity units consumed: 200
Debug: Units = 200
Debug: Rate per unit = 5.00
Debug: Calculated Bill = 1000.00
Total Electricity Bill = 1000.00


=== Code Execution Successful ===
```

Result

Thus, the program was executed successfully and the macro displayed the intermediate values, making the program easier to debug.

Experiment 14: Minimal Memory Design

Aim

To write a C program to calculate the total amount to be paid by using the smallest suitable data types.

Algorithm

1. Start the program.
2. Read the cost of an item and the number of items.
3. Calculate the total amount.
4. Display the total amount.
5. Stop.

Program

```
#include <stdio.h>
int main()
{
   unsigned short cost;
   unsigned char quantity;
   unsigned int total;
   printf("Enter cost of one item: ");
   scanf("%hu", &cost);
   printf("Enter number of items: ");
   scanf("%hhu", &quantity);
   total = cost * quantity;
   printf("Total Amount = %u\n", total);
   return 0;
}
```

Output

main.c

```c
#include <stdio.h>

int main()
{
    unsigned short cost;
    unsigned char quantity;
    unsigned int total;

    printf("Enter cost of one item: ");
    scanf("%hu", &cost);

    printf("Enter number of items: ");
    scanf("%hhu", &quantity);

    total = cost * quantity;

    printf("Total Amount = %u\n", total);

    return 0;
}
```

Output

```
Enter cost of one item: 45
Enter number of items: 6
Total Amount = 270


=== Code Execution Successful ===
```

Result

Thus, the program was executed successfully. Using the smallest suitable data types helps optimize memory usage while producing the correct result.

Experiment 15: Program Modification Task

Aim

To write a C program to convert Fahrenheit to Celsius using macros and appropriate data types.

Algorithm

1. Start the program.
2. Define the constant value using a macro.
3. Read the temperature in Fahrenheit.
4. Convert Fahrenheit to Celsius using the formula:

$$\text{Celsius} = (\text{Fahrenheit} - 32) \times \frac{5}{9}$$

5. Display the temperature in Celsius.
6. Stop.

Program

```
#include <stdio.h>

#define OFFSET 32
#define FACTOR 5.0/9.0

int main()
{
   float fahrenheit, celsius;

   printf("Enter temperature in Fahrenheit: ");
   scanf("%f", &fahrenheit);
```

```
    celsius = (fahrenheit - OFFSET) * FACTOR;

    printf("Temperature in Celsius = %.2f\n", celsius);

    return 0;
}
```

Output

Result

Thus, the program was executed successfully. Using macros for constants and the float data type produced an accurate Fahrenheit-to-Celsius conversion.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementatio	Correct implementati	Basic implementation with	Incorrect or incomplete

		n with optimal solution	on with minor issues	errors or inefficiencies	implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation